



Colour Models and Colour Spaces

Romey Bajada (0014306H), Mia Busuttil (0150406L), Julia-Kay Gutiza (0170706L),
Kyra Talbot (0391806L), Rebecca Hayward (0148793A)
B.Sc. (Hons) Information Technology (Artificial Intelligence)

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as "the unacknowledged use, as one's own, of work of another person, whether or not such work has been published, and as may be further elaborated in Faculty or University guidelines" (University Assessment Regulations, 2009, Regulation 39 (b)(i), University of Malta).

We, the undersigned, declare that the assignment submitted is our work, except where acknowledged and referenced.

We understand that the penalties for committing a breach of the regulations include loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected and will be given zero marks.

(N. B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Romey Bajada

Student Name



Signature

Mia Busuttil

Student Name

mimi

Signature

Kyra Talbot

Student Name



Signature

Julia-Kay Gutiza

Student Name



Signature

Rebecca Hayward

Student Name



Signature

ARI2129

Course Code

Colour Models and Colour Spaces

Title of work submitted

18-05-2026

Date

Contents

1. Introduction.....	4
2. RGB.....	4
2.1 What is RGB?.....	4
2.2 How RGB works.....	5
2.3 Technical Characteristics of RGB.....	5
2.4 Real World Applications of RGB.....	5
2.5 Converting between Colour Spaces.....	5
3. HSV.....	6
3.1. What is HSV.....	6
3.2 The Problem HSV Solves.....	6
3.3 The Geometry of HSV.....	6
3.4 The Edge Case: When $S = 0$	6
3.5 Converting to and from HSV.....	7
3.5.1 Worked Example: RGB -> HSV -> RGB.....	8
3.6 Real-World Applications of HSV.....	8
3.7 Failure Cases and When not to use HSV.....	9
4. YCbCr.....	9
4.1 Introduction.....	9
4.2 Core Components of YCbCr.....	9
4.3 Why YCbCr is Used.....	9
4.4 Converting To and From YCbCr Through RGB.....	9
4.5 Chroma Subsampling.....	10
4.6 Skin Detection using YCbCr.....	10
4.7 Real-World Applications of YCbCr.....	11
5. LAB.....	11
5.1 What is LAB?.....	11
5.2 The Problem LAB solves.....	11
5.3 Analogy : A Ruler.....	12
5.4 The Key Insight: L^* is separate from colour.....	12
5.5 Delta E (ΔE) - measuring colour difference.....	12
5.6 Converting to and from LAB.....	12
5.7 Real-World Applications of LAB.....	13
6. Conclusion.....	13
7. Cheat Sheet.....	13
7.1 Comparisons.....	13
7.2 Conversions.....	14
8. Glossary.....	15
9. References.....	16

Github url: https://github.com/romeybajada/cv_colour_models

1. Introduction

Colour models and colour spaces are essential in digital imaging and computer vision because they allow computers to represent, process and interpret colour information mathematically. Different colour spaces are designed for different purposes: RGB is used for image capture and display, HSV provides a more human-intuitive representation of colour, YCbCr improves compression efficiency by separating brightness from colour information, and LAB models colour based on human perception. In computer vision and artificial intelligence, choosing the correct colour space is important because it directly affects tasks such as object detection, segmentation, compression, and image enhancement. These notes explore the structure, characteristics, conversions, advantages, limitations, and real-world applications of these four big colour spaces.

2. RGB

2.1 What is RGB?

RGB is a colour model that represents colours as a combination of three primary components: Red, Green and Blue. This colour model primarily exists as it provides a practical bridge between human visual perception, digital hardware and standardised communication. Humans can distinguish thousands of colour shades and intensities however, our eyes only perceive two dozen shades of grey, RGB mimics the way our eyes perceive colour to provide a richer data set for visual information. RGB is also used for object identification as it simplifies the process of identifying and extracting specific objects for a complex scene. It is the most common way for digital devices to handle colour; for example, scanners capture light in RGB, software applications process data in RGB.

RGB spaces are categorised, each version represents a different stage of the digital workflow:

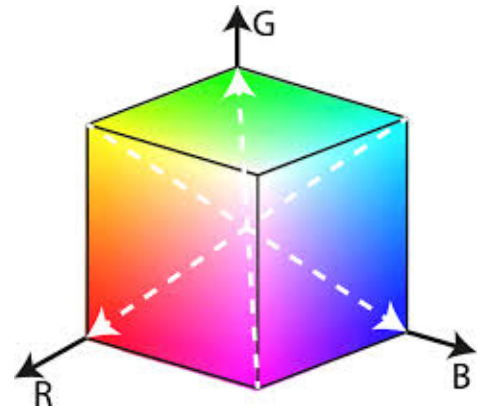
RGB Space	Definition	Example
Sensor RGB	Device specific data captured by a camera or scanner. Defined by spectral sensitivities of the device's hardware	Raw data captured by a digital camera or scanner
Unrendered RGB	Device independent estimate of the original scenes colourimetry, its purpose is to preserve the original colour information for future processing	ISO RGB
Rendered RGB	Designed to match the characteristics of output devices like monitors	sRGB, Adobe RGB 98, Apple RGB

Analogy: The Three Spotlights

Imagine you are standing in a pitch black room facing a white wall. To create colour on that wall, you have three theatrical spotlights: one Red, one Green and one Blue. Each light has a dimmer switch with 256 levels, 0 for off and 255 for full brightness. With these applications the wall can recreate almost any colour in nature and put on the most amusing show.

2.2 How RGB works

The model is based on a **Cartesian coordinate system** where each colour is defined by its primary spectral components: Red, Green, Blue. An RGB image is made of three separate grayscale-like images (one for each primary). When these images are fed into a monitor, they combine on a phosphor screen to create the final composite colour image we see. Each pixel's colour is specified by three intensity values. In a standard 24-bit true colour image, each of the three channels uses 8 bits therefore the mathematical combination allows for 16,777,216 distinct colours.



2.3 Technical Characteristics of RGB

The RGB model is defined as a geometric representation of colours within a three-dimensional Cartesian coordinate system. A drawback of this model is that its channels are highly correlated, meaning that changes in one channel often impact the others, which makes it less efficient for certain processing tasks like image compression. Additionally, RGB suffers from perceptual non-uniformity because it does not perfectly match the sensitivity of human vision; consequently, a mathematical change in a colour value may be more or less noticeable to an observer depending on where that colour falls within the spectrum.

2.4 Real World Applications of RGB

1. Digital Archiving and Conservation: For high end archiving images are often stored as raw sensor data, this preserves the original scene's colourimetry
2. Web design: sRGB serves as the default colour space for the internet and multimedia applications
3. Professional Image Editing: Working spaces like Adobe RGB 998 and ROMM RGB are used in software like Photoshop and editing

2.5 Converting between Colour Spaces

Colour space conversion is the mathematical process of translating colour data from one representation to another. Each colour space in this document was designed for a specific purpose, and moving between them requires precise equations, as using the wrong formula or skipping a step introduces errors ranging from subtle colour shifts to completely incorrect output.

RGB acts as the **universal hub**; there are no direct formulas between HSV, YCbCr, and LAB. Every path between non-RGB spaces passes through RGB first, because all alternative spaces are defined as transformations from RGB, not from each other. Think of it like temperature scales where Celsius and Fahrenheit describe the same thing in different terms, and converting between them requires an exact formula. The wrong formula gives a number that looks valid but is physically incorrect. Colour conversions work the same way.

From	To	Route
HSV	LAB	HSV -> RGB -> XYZ -> LAB

YCbCr	HSV	YCbCr -> RGB -> HSV
LAB	YCbCr	LAB -> XYZ -> RGB -> YCbCr

3. HSV

3.1. What is HSV

HSV describes a colour using the following three channels:

Channel	Definition	Range
H - Hue	What colour it is (red, green, yellow)	0°– 360°
S - Saturation	How vivid vs washed out it is	0- 1
V - Value	How bright vs dark it is	0 - 1

Analogy: Think like a painter - H is which paint colour you picked, S is how much water you added, and V is how much black you mixed in. To darken without changing the colour, just reduce V.

3.2 The Problem HSV Solves

RGB tangles colour, vividness and brightness across three channels simultaneously. (100, 149, 237) is a vivid blue but nothing in those numbers tells you what the colour looks like intuitively. Alvy Ray Smith designed HSV in 1978 so artists could think "bright, saturated blue" instead of hunting through RGB values. RGB describes colour the way hardware thinks. HSV describes it the way humans think.

3.3 The Geometry of HSV

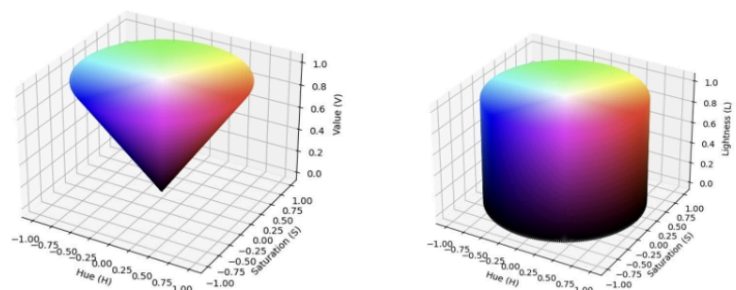
HSV is typically represented as a cylinder:

- **Vertical axis = Value:** bottom is black (V=0), top is maximum brightness (V=1)
- **Distance from the central axis = Saturation:** centre is grey (S=0), outer edge is full vivid colour (S=1)
- **Angle around the cylinder = Hue:** measured in degrees from 0° to 360°

A cone is sometimes used instead since at V = 0 all colours collapse to black and at S = 0 all hues collapse to grey. However the cylinder is standard for computational convenience.

3.4 The Edge Case: When S = 0

When saturation is zero the pixel is **achromatic**. The hue formula requires dividing by $\delta = M - m$. When S = 0, δ is also zero, causing a division by zero. By convention, implementations set H = 0° in this case. This becomes a practical bug in computer vision pipelines: if your hue threshold includes 0°, near-grey



pixels will pass the mask not because they match your target colour, but because the conversion formula assigned them an arbitrary hue.

Fix in OpenCV: always pair 'cv2.inRange()' with $S_{\min} \geq 30$ to exclude unreliable hue pixels

3.5 Converting to and from HSV

HSV is defined as a transform of RGB. The only path into HSV goes through RGB first.

Assume R, G, B are normalised to [0, 1].

Method 1: Standard Real-Valued (Floating-Point) Algorithm

This is the classic, widely-used approach.

Step 1 – Find extremes:

$$M = \max(R, G, B), m = \min(R, G, B), \delta = M - m$$

Step 2 – Assign Value (V):

$$V = M$$

Step 3 – Calculate Saturation (S):

$$S = \begin{cases} 0 & \text{if } V = 0 \\ \frac{\delta}{V} & \text{otherwise} \end{cases}$$

Step 4 – Calculate Hue (H):

$$H = \begin{cases} 0^\circ & \text{if } \delta = 0 \text{ (achromatic)} \\ 60^\circ \times \left(\frac{G - B}{\delta} \bmod 6 \right) & \text{if } V = R \\ 60^\circ \times \left(\frac{B - R}{\delta} + 2 \right) & \text{if } V = G \\ 60^\circ \times \left(\frac{R - G}{\delta} + 4 \right) & \text{if } V = B \end{cases}$$

If $H < 0^\circ$, add 360° .

Method 2: Accurate Integer-Based Algorithm (Lossless)

For embedded systems and FPGAs without floating-point hardware, an integer-based method can convert RGB to HSV while still *perfectly recovering the original RGB values*. Instead of decimal calculations, it uses fixed-point integer maths. The algorithm first finds the maximum, minimum, and middle RGB values, then uses their order to determine the hue region. Saturation and hue are calculated using scaled integer operations instead of normal division. This method is faster than

floating-point conversion and works well on hardware-focused systems. Unlike floating-point, it guarantees lossless round-trip conversion, the original RGB values can be recovered exactly.

Method 3: Fast Lookup Table (LUT) Method

For real-time video tasks like fire detection, speed is more important than perfect accuracy. The LUT method speeds up HSV conversion by replacing expensive division operations with a precomputed lookup table. This reduces the calculation mostly to multiplication and bit-shifting, making it the fastest method. However, because the table only stores approximate values, some colour accuracy is lost, especially for highly saturated or extreme colours.

3.5.1 Worked Example: RGB -> HSV -> RGB

Input: A vivid orange – R = 255, G = 140, B = 0

Step 1 - Normalise: R = 1.000, G = 0.549, B = 0.000

Step 2 - Find extremes:

$$M = 1.000, \quad m = 0.000, \quad \delta = 1.000 - 0.000 = 1.000$$

Step 3 - Value:

$$V = 1.000 \quad (100\%)$$

Step 4 - Saturation:

$$S = \frac{1.000}{1.000} = 1.000 \quad (100\%)$$

Step 5 - Hue (V = R, so use first formula):

$$H = 60^\circ \times \left(\frac{0.549 - 0.000}{1.000} \bmod 6 \right) = 60^\circ \times 0.549 = 32.9^\circ$$

Result: HSV = (33°, 100%, 100%); a fully saturated, fully bright orange.

For the inverse, HSV -> RGB, two methods can be used. The **Standard Real-Valued Inverse** works by dividing the hue into six 60° sectors and assigning RGB values based on which sector the colour falls in, then adjusting the brightness using the value and saturation. The **Lossless Integer-Based Inverse** reverses the forward conversion exactly using only integer operations, recovering the three RGB components from the stored hue, saturation, and value without any rounding loss.

3.6 Real-World Applications of HSV

- **Colour pickers in design software** (Photoshop, Figma) – because users think in hue and saturation, not red/green/blue levels.
- **Complementary colour detection** – complementary colours are always exactly 180° apart on the hue axis; expressing it in RGB requires significantly more computation.
- **Object detection in computer vision** – isolating a specific colour (e.g., detecting a red traffic light) is far easier in HSV because you just threshold the Hue channel.
- **Fire detection in video** – real-time systems use the LUT method for speed.
- **Medical imaging** – colour coding tissue types by saturation level.

3.7 Failure Cases and When not to use HSV

HSV is a powerful tool for colour-based computer vision, but it fails predictably in several situations.

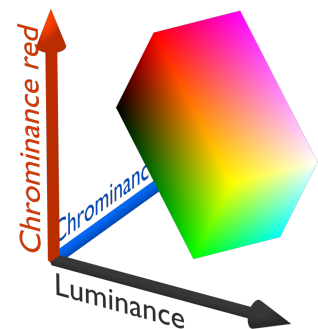
Situation	Better alternative	Why
Perceptual colour similarity measurement	LAB (CIELAB)	Euclidean distance in LAB correlates with perceived difference; HSV distance does not
Compression and transmission	YCbCr	Designed for chroma subsampling; HSV is not
Shadows and specular highlights	LAB L* channel	Perceptually linearised lightness is more robust than V
Near-grey or pastel objects	RGB	No colour space handles near-grey well; learned features outperform hand-tuned thresholds

4. YCbCr

4.1 Introduction

YCbCr is a colour space widely used in digital video and image compression systems such as JPEG and MPEG. Unlike RGB, it separates brightness (luminance) from colour information (chrominance), making it efficient for compression and transmission.

Analogy - YCbCr can be thought of as separating a black-and-white image from its colour information. Luminance provides structure and detail, while chrominance adds colour with reduced precision, thereby improving compression efficiency.



YCbCr is effective because the human eye is more sensitive to brightness detail than colour detail, allowing colour information to be stored at lower precision with minimal visible quality loss.

4.2 Core Components of YCbCr

YCbCr represents an image using three components: **Y'** (luma), which represents brightness and carries most image detail, and **Cb** (blue chroma) and **Cr** (red chroma), which represent blue and red chroma differences (differences between the blue/red components and the brightness).

4.3 Converting To and From YCbCr Through RGB

Images are commonly converted from RGB to YCbCr before compression because YCbCr supports chroma subsampling, reducing file size with minimal visible quality loss. The conversion separates luminance (Y) from chrominance (Cb and Cr) using weighted combinations of RGB values. The standard ITU-R BT.601 conversion is:

$$Y = 0.299R + 0.587G + 0.114B + 16$$

$$Cb = -0.1687R - 0.3313G + 0.500B + 128$$

$$Cr = 0.500R - 0.4187G - 0.0813B + 128$$

The +128 centres chrominance for 8-bit storage. **Matrix Form (for efficient computation):**

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5000 \\ 0.5000 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 16 \\ 128 \\ 128 \end{bmatrix}$$

The reverse BT.601 transformation converts YCbCr back to RGB using:

$$R = 1.000(Y - 16) + 1.402(Cr - 128)$$

$$G = 1.000(Y - 16) - 0.344(Cb - 128) - 0.714(Cr - 128)$$

$$B = 1.000(Y - 16) + 1.772(Cb - 128)$$

In BT.601, Y ranges from 16–235 rather than 0–255 to provide headroom and footroom in the digital signal. That is why we add and subtract 16 with Y.

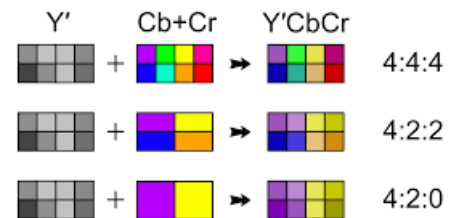
Matrix form:

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.000 & 0.000 & 1.402 \\ 1.000 & -0.344 & -0.714 \\ 1.000 & 1.772 & 0.000 \end{bmatrix} \begin{bmatrix} Y - 16 \\ Cb - 128 \\ Cr - 128 \end{bmatrix}$$

The constants (1.402, 0.344, 0.714, 1.772) are derived from the reciprocals of the original luminance weighting coefficients.

4.4 Chroma Subsampling

Chroma subsampling reduces chrominance resolution to save storage and bandwidth while preserving luminance detail. Brightness changes are very noticeable to the human eye, but small colour changes are much harder for us to detect. So instead of storing full colour information for every single pixel, we reduce the resolution of the colour channels while keeping the brightness channel fully detailed.



The notation used to describe subsampling follows the form **J:a:b**. The first number represents luminance samples, while the second and third numbers represent chrominance samples.

- In 4:4:4 every pixel has its own full colour information, so nothing is reduced — this is the highest quality but also the largest file size.
- In 4:2:2 the colour information is reduced horizontally. This means pairs of pixels share some colour data side by side, but brightness is still stored for every pixel.
- In 4:2:0 the colour information is reduced both horizontally and vertically. So a group of pixels shares the same colour information, while each pixel still keeps its own brightness.

4.5 Skin Detection using YCbCr

YCbCr is often used for skin detection because skin tones fall within a narrow range of **Cb and Cr values**, while brightness (Y) varies with lighting. Separating luminance from chrominance makes detection more robust to illumination changes.

Skin detection is done by **thresholding** the Cb and Cr channels. However, this method can incorrectly detect non-skin regions like hair or shadows since it relies only on colour similarity, not context.

Despite this, the technique is still widely used in applications such as face detection and gesture recognition because it is fast and computationally efficient.

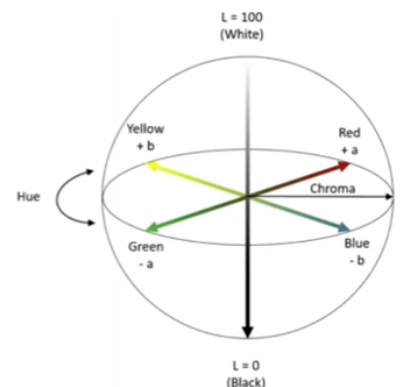
4.6 Real-World Applications of YCbCr

- **JPEG compression:** Cb and Cr channels are downsampled (typically 4:2:0), reducing file size with minimal visible quality loss.
- **MPEG/H.264/H.265 codecs:** Apply the same principle to video frames for efficient compression.
- **Digital television:** BT.601 (SD) and BT.709 (HD) are standard broadcast colour formats.
- **Fire detection:** Fire pixels exhibit characteristic YCbCr patterns useful for CV systems.

5. LAB

5.1 What is LAB?

LAB (formally written as **CIE L*A*B*** or **CIELAB**) is a colour space originally created by the International Commission on Illumination (CIE) in 1976. While RGB was designed around how display hardware works, and YCbCr was designed around compression efficiency, LAB was designed around a single goal: to match the way **humans perceive colour**.



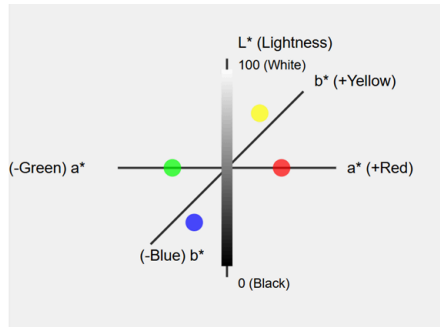
LAB represents every colour using three channels:

Channel	Definition	Range
L* - Lightness	How bright or dark the pixel is	0 (black) to 100 (white)
A* - Green-Red axis	How green or red the pixel is	-128(green) to +127(red)
B* - Blue-Yellow axis	How blue or yellow the pixel is	-128(blue) to +127(yellow)

Unlike RGB, these axes map directly onto how the human brain processes colour, using opponent signals (light/dark, red/green, yellow/blue) rather than mixing light intensities. At zero on both A* and B*, the colour is a neutral grey. The further a value moves from zero on either axis, the more vivid and colourful the pixel becomes.

5.2 The Problem LAB solves

RGB describes colour the way a **camera sensor** thinks, as intensities of three light wavelengths. The problem is that its numbers have no reliable relationship to what a human observer actually perceives.



LAB was designed to fix exactly this. Its defining property is called **perceptual uniformity**: a given numerical change anywhere in LAB space corresponds to a roughly equal perceived change in colour. In practical terms: **distance in LAB = visual difference**.

This makes LAB the colour space of choice for any task involving colour comparison, correction, or segmentation, and the official connection space used inside ICC colour management profiles, the system that ensures colours on your

screen match what a printer outputs.

5.3 Analogy : A Ruler

Think of LAB like a ruler with evenly spaced marks, versus RGB which is a ruler where the marks are unevenly spaced. On the RGB ruler, a gap of “10 units” might represent a barely-visible difference in one place and a very obvious difference somewhere else. On the LAB ruler, every gap of “10 units” looks the same size to your eyes, wherever on the ruler you are. Another way to think about it would be that RGB describes colour the way a hardware engineer thinks, in terms of voltages and subpixels while LAB describes colour the way a human thinks, in terms of brightness, and colours.

5.4 The Key Insight: L* is separate from colour

This is the most practically important property of LAB. **Brightness is completely separated from colour information.**

- In **RGB**: to brighten an image, you must increase R,G, and B together (the colour shifts too)
- in **LAB**: you increase L* only, the A* and B* channels (and therefore the colour) are completely untouched

This is why LAB is the standard choice for contrast adjustment, selective sharpening, and any task where brightness and colour need to be manipulated independently.

5.5 Delta E (ΔE) - measuring colour difference

Once two colours are expressed in LAB, their perceptual difference can be measured as a simple straight-line distance in 3D space. This metric is called **Delta E (ΔE)**.

$$\Delta E = \sqrt{(\Delta L^*)^2 + (\Delta a^*)^2 + (\Delta b^*)^2}$$

5.6 Converting to and from LAB

RGB to LAB requires two transformations in sequence: **RGB -> CIE XYZ -> LAB** because there is no direct path between them. XYZ is a device-independent intermediate space that acts as the bridge.

First, gamma encoding is removed from each channel to convert display values into physically proportional light measurements. The **linearised values** are multiplied by the sRGB-to-XYZ matrix,

then normalised against the D65 white point. A nonlinear perceptual mapping $f(t)$ is then applied to each normalised coordinate:

$$f(t) = t^{\frac{1}{3}} \quad \text{if } t > 0.008856,$$

$$\text{otherwise } f(t) = 7.787t + \frac{16}{116}$$

This models how humans perceive lightness non-linearly. The final LAB values are:

$$L^* = 116 \cdot f(y) - 16$$

$$a^* = 500 \cdot (f(x) - f(y))$$

$$b^* = 200 \cdot (f(y) - f(z))$$

The inverse reverses all five steps. Note that the XYZ-to-RGB matrix can produce slightly negative values for colours near the sRGB gamut boundary. These must be clipped to zero before applying the gamma power function.

5.7 Real-World Applications of LAB

- **Professional printing (ICC colour profiles)** - measuring and compensating for how different printers/monitors render colour.
- **Colour difference measurement (ΔE)** - the standard formula for "how different do these two colours look?" used in manufacturing and quality control (e.g., ensuring a car's paint matches across parts).
- **Image quality assessment** - computing perceptual distances between a compressed and original image.
- **Colour correction tools** - LAB is used in Photoshop's Curves adjustments because its channels map directly to perceptual attributes.
- **Medical/scientific imaging** - standardised colour representation independent of the capture device.

6. Conclusion

Colour spaces are fundamental to digital imaging, computer vision, and artificial intelligence because they provide different ways of representing and processing visual information. This documentation explored four major colour spaces RGB, HSV, YCbCr, and LAB, each designed for a specific purpose and each offering unique advantages and limitations. RGB remains the standard for image capture and display due to its simplicity and direct relationship with hardware, while HSV provides a more intuitive representation for colour-based segmentation and object detection. YCbCr is optimised for image and video compression through the separation of luminance and chrominance, making it essential in multimedia systems, whereas LAB focuses on perceptual uniformity and accurate colour comparison.

It was also demonstrated that colour space conversion is a critical process in image processing pipelines, with RGB acting as the central intermediary between most models. Overall, no single colour space is universally superior; the most effective choice depends on the requirements of the application, including computational efficiency, perceptual accuracy, robustness to lighting variation and compression needs.

7. Cheat Sheet

7.1 Comparisons

Colour Space	Main Purpose	Key Advantage	Key Weakness	Typical Application
RGB	Hardware colour representation	Simple, universal, native to all displays and cameras	Mixes colour and brightness in every channel; perceptually non-uniform	Display rendering, image capture, web design
HSV	Human-intuitive colour description and segmentation	Separates hue from brightness; robust to moderate lighting change	Hue undefined at low saturation; fails on near-grey, specular highlights, and deep shadows	Colour thresholding, object detection, colour pickers
YCbCr	Compression and broadcast transmission	Separates luminance from chrominance; supports chroma subsampling	Not designed for segmentation; perceptually non-uniform	JPEG, MPEG, H.264/H.265, digital television, skin detection
LAB	Perceptual colour measurement	Perceptually uniform: equal numerical distances equal perceived differences	Computationally expensive; requires XYZ intermediate step; unintuitive axes	Colour difference (ΔE), ICC profiles, quality control, Photoshop Curves

7.2 Conversions

Conversion	Key Steps	Output Ranges
RGB -> HSV	$V = \max(R, G, B)$ $S = \delta/V$ $H = 60^\circ \times \text{formula based on which channel is max}$	H: 0 – 360° S: 0 – 1 V: 0 – 1
HSV -> RGB	$C = V \times S$	R: 0 – 255

	• assign (R,G,B) from hue sector table • add $m = V - C$ to each	G: 0 – 255 B: 0 – 255
RGB -> YCbCr	• $Y = 0.299R + 0.587G + 0.114B$ • $Cb = -0.1687R - 0.3313G + 0.5B + 128$ • $Cr = 0.5R - 0.4187G - 0.0813B + 128$	Y: 16–235 Cbr: 16–240 Cr: 16–240
YCbCr -> RGB	• $R = (Y-16) + 1.402(Cr-128)$ • $G = (Y-16) - 0.344(Cb-128) - 0.714(Cr-128)$ • $B = (Y-16) + 1.772(Cb-128)$	R: 0 – 255 G: 0 – 255 B: 0 – 255
RGB -> LAB	Normalise -> remove gamma -> multiply by XYZ matrix -> divide by D65 white point -> apply $f(t) = t^{1/3}$ -> $L^* = 116f(y) - 16$, $a^* = 500(f(x) - f(y))$, $b^* = 200(f(y) - f(z))$	L*: 0–100 a*: -128 – +127 b*: -128 – +127
LAB -> RGB	Recover x,y,z from L^*, a^*, b^* -> apply $f^{-1}(t) = t^3$ -> multiply by D65 -> multiply by inverse XYZ matrix -> apply gamma -> scale to 0–255	R: 0 – 255 G: 0 – 255 B: 0 – 255

8. Glossary

Achromatic – A colour with no hue (white, grey, black). Delta = 0 in HSV.

Chroma (C) – In HSV->RGB: $C = V \times S$. The "amount" of colour present.

Chrominance (Cb, Cr) – The colour-difference components in YCbCr, encoding how far a colour deviates from neutral grey.

Colour model – A conceptual framework that describes how colours can be represented using a set of numerical components

Colour space – A mathematical model that represents colours as coordinates in a defined system (e.g., RGB, HSV, LAB).

Complementary colours – pairs of colours that sit directly opposite each other on the colour wheel, 180° apart in hue.

D65 illuminant – The standard reference white point representing average midday daylight. Used in sRGB and CIELAB.

Delta (δ) – The difference between the maximum and minimum RGB component. Measures "how colourful" a pixel is. Zero means grey.

Fixed-point arithmetic – A method of representing fractional numbers using integers by scaling by a power of 2. Avoids floating-point units on embedded hardware.

Gamma correction – A non-linear encoding applied to RGB values so that the perceived brightness on a display is linear. Must be removed before linear colour operations.

Hue (H) – The "type" of colour, represented as an angle (0°–360°) on the colour wheel.

Integer lossless conversion – A conversion algorithm using only integer operations that guarantees the original values can be perfectly recovered on inversion (no rounding loss).

ITU-R BT.601 – The international standard defining RGB↔YCbCr conversion for standard-definition digital television.

LUT (Lookup Table) – A precomputed table that replaces a slow mathematical operation (like division) with a fast table lookup. Used for real-time colour conversion.

Luminance (Y or L*) – A measure of perceived brightness. In YCbCr: a weighted sum of RGB. In LAB: a perceptually linearised brightness scale.

Perceptual uniformity – A property of a colour space where equal numerical distances correspond to equal perceived colour differences. CIELAB aims for this; RGB does not.

Saturation (S) – How vivid or pure a colour is. 0 = grey, 1 = fully saturated.

Value (V) – In HSV: the maximum brightness of a colour. 0 = black, 1 = full brightness.

XYZ (CIE XYZ) – A device-independent intermediate colour space used as a bridge in RGB↔LAB conversion. Based on human cone response curves.

RGB (Red, Green, Blue) – A colour model that represents colours by combining red, green, and blue light at different intensities.

Red, Green, Blue Channels – The three primary colour components used to create every colour in an RGB image.

Colour Model – A system used to represent and organise colours numerically.

Sensor RGB – Device-specific raw colour data captured directly by a camera or scanner sensor.

Unrendered RGB – A device-independent RGB format designed to preserve original colour information before editing or display.

Rendered RGB – RGB data processed for display on output devices such as monitors or printers.

sRGB – The standard RGB colour space commonly used for websites, monitors, and multimedia applications.

9. References

[1] Muratbekova, M., Toganis, N., Igali, A., Shagyrov, M., Kadyrgali, E., Yerkin, A., & Shamoï, P. (2026). Color models in image processing: a review and experimental comparison. *Discover Applied Sciences*, 8, 494.

URL: <https://doi.org/10.1007/s42452-025-08192-7>

Summary: This review surveys a wide range of colour models and spaces, analysing their theoretical foundations, computational properties, and real-world applications. The authors cover traditional models such as RGB and CMYK, perceptually uniform spaces including CIELAB and CIELUV, and fuzzy-based approaches. A series of experiments evaluates models from multiple perspectives (device dependency, chromatic consistency, and computational complexity). Results show that the HS* family (HSV, HSL, HSI) is most closely aligned with human perception, while CIE spaces offer better perceptual uniformity at higher computational cost. The paper also identifies open research challenges and future directions in colour representation.

Relevance: Provides a comprehensive comparison of the exact colour spaces covered in this learning pack, making it ideal for understanding why each space exists and what its practical trade-offs are relative to alternatives.

[2] Ghanem, S., & Holliman, J. H. II (2024). Impact of color space and color resolution on vehicle recognition models. *Journal of Imaging*, 10(7), 155.

URL: <https://doi.org/10.3390/jimaging10070155>

Summary: This paper investigates how colour space selection and colour resolution affect the performance of machine learning models for vehicle recognition. The authors train models on a dataset captured under diverse lighting conditions, and compare linear and nonlinear colour mappings across RGB, HSV, LAB, and YCbCr. RGB's mixing of colour and intensity information is identified as a key limitation under variable illumination. Results show that colour space choice significantly influences recognition accuracy, while model performance degrades only gradually as colour resolution is reduced, suggesting that simpler representations can remain effective. The study also identifies potential for automatic colour space selection through feature learning.

Relevance: Provides direct evidence of what happens when you choose the wrong colour space in a real computer vision pipeline, covering all four spaces in this learning pack.

[3] Z.-G. Liu, S.-Y. Du, Y. Yang, and X.-H. Ji, "A fast algorithm for color space conversion and rounding error analysis based on fixed-point digital signal processors," *Computers & Electrical Engineering*, vol. 40, no. 4, pp. 1405–1414, May 2014, doi: <https://doi.org/10.1016/j.compeleceng.2013.01.011>.

URL: <https://www.sciencedirect.com/science/article/abs/pii/S0045790613000177?via%3Dihub>

Summary: This paper presents a fast algorithm for converting YCbCr images to HSV using fixed-point arithmetic instead of floating-point calculations. The method replaces expensive multiplications with shifts, lookup tables, and simple integer operations, making it suitable for DSP hardware and real-time systems. The authors explain that direct conversion from YCbCr to HSV is not possible without RGB as an intermediate step. The paper also studies rounding errors produced during scaling and shifting operations and introduces a small lookup-table correction method to improve accuracy. Experimental results show the algorithm achieves much faster performance while maintaining accuracy suitable for applications such as skin and flame detection.

Relevance: Primary source for the RGB-as-hub explanation, the LUT method description, rounding error analysis, and the justification for why the floating-point method is used in the notebook.

[4] V. Chernov, J. Alander, and V. Bochko, "Integer-based accurate conversion between RGB and HSV color spaces," *Computers & Electrical Engineering*, vol. 46, pp. 328–337, Aug. 2015,

URL: <https://doi.org/10.1016/j.compeleceng.2015.08.005>

Summary: This paper proposes a lossless integer-based method for RGB to HSV colour conversion that avoids floating-point arithmetic entirely. Unlike classical methods, the algorithm guarantees that original RGB values can be perfectly reconstructed after conversion, preventing information loss caused by rounding errors. The approach relies only on integer arithmetic and bitwise operations,

making it efficient for hardware-constrained systems. The paper derives formulas for hue sectors, saturation calculations, and a high-precision integer hue representation. Experimental comparisons show the method runs about twice as fast as standard floating-point implementations while preserving exact reversibility. The authors suggest it as a practical replacement for traditional RGB <-> HSV conversion methods.

Relevance: Primary source for the integer-based RGB to/from HSV method described in the notes (Method 2), the lossless round-trip guarantee concept, and the performance comparison between floating-point and integer approaches that justifies the method choice in the notebook.

[5] M. Emre. Celebi, H. A. Kingravi, and F. Celiker, "Fast colour space transformations using minimax approximations," *IET Image Processing*, vol. 4, no. 2, pp. 70–80, Apr. 2010,
URL: <https://doi.org/10.1049/iet-ipr.2008.0172>

Summary: This paper introduces Minimax Approximations for Color-space Transformations (MACT), a technique for speeding up nonlinear colour space conversions using rational function approximations. The method is applied to RGB conversions into CIELAB, HSI, and SCT colour spaces. The authors explain that CIELAB is valuable because of its perceptual uniformity, while HSI and SCT are useful for intuitive colour manipulation and illumination-invariant image analysis. The paper compares MACT with lookup-table interpolation methods in terms of speed, memory use, implementation difficulty, and accuracy. Results show that minimax approximations provide efficient and accurate colour conversions, making them useful for high-performance image processing systems and real-time applications.

Relevance: Used to support the RGB to LAB pipeline discussion, the cube-root approximation reference in the f(t) step, and the explanation of why LAB's perceptual uniformity makes it suitable for colour difference measurement and segmentation tasks.

[6] Smith, A. R. (1978). Color gamut transform pairs. *ACM SIGGRAPH Computer Graphics*, 12(3), 12–19.
URL: <https://doi.org/10.1145/965139.807361>

Summary: This seminal paper explores various mathematical models of the RGB monitor gamut, specifically focusing on the derivation of algorithms for transforming the standard RGB "colorcube" into more perceptually intuitive spaces. The author introduces and details two primary models: the hexcone model (HSV), which mimics how artists mix paints using hue, saturation, and value; and the triangle model (HSL), which generalizes color based on hue, saturation, and brightness. The research evaluates these transformations based on their computational efficiency, noting that the hexcone model is particularly fast because it avoids expensive trigonometric functions. Furthermore, the paper discusses the NTSC (IQY) space as a psychophysical adjustment of RGB that accounts for human perception and phosphor properties.

Relevance: This source is foundational for understanding the mathematical bridge between hardware-centric RGB data and the human-centric variables of hue, saturation, and lightness. It provides the specific transform algorithms used in computer graphics and painting programs to make digital color control more accessible to users while maintaining computational speed.

[7] Kumar, T., & Verma, K. (2010). A Theory Based on Conversion of RGB image to Gray image. *International Journal of Computer Applications*, 7(2), 7–10.
URL: <https://doi.org/10.5120/1140-1493>

Summary: This paper explores the fundamental motivations for using color in image processing, primarily that humans can discern thousands of color shades compared to only about two dozen shades of gray. It defines the technical properties of the RGB model, including its Cartesian coordinate system structure, 24-bit pixel depth, and its representation as three distinct component images. The core of the research proposes a specific mathematical transformation equation— $I_y = 0.333F_r + 0.5F_g + 0.1666F_b$ —to convert color data into grayscale intensities. Through MATLAB-based experiments, the authors demonstrate that their method provides a more accurate conversion with a lower percentage error than the established Braun method.

Relevance: This source is essential for understanding the biological and mathematical foundations of the RGB model. It provides the specific logic for how digital devices handle high-depth "true color" and the practical techniques used to simplify that data for tasks like edge detection or medical imaging.

[8] Koju, R., & Joshi, S. R., Comparative analysis of color image watermarking technique in RGB, YUV, and YCbCr color channels, Nepal Journal of Science and Technology, vol. 15, no. 2, pp. 133–140, 2014.

URL: [Skin Segmentation Using YCBCR and RGB Colour Models](#)

Summary

This paper looks at how skin segmentation can be done using RGB and YCbCr colour models, which is an important step for tasks like face detection. It compares both approaches and shows how YCbCr works better because it separates brightness (luminance) from colour (chrominance), making it easier to isolate skin tones. The method uses chrominance thresholding and statistical modelling to detect skin pixels across different lighting conditions and skin types. While both RGB and YCbCr can be used for segmentation, YCbCr gives more consistent and accurate results. The study concludes that separating luminance from chrominance improves detection reliability in real-world image processing applications.

Relevance

This paper supports the skin detection section by explaining why YCbCr is more effective than RGB for identifying skin regions. It directly relates to the implementation using Cb and Cr thresholding in the code walkthrough.